



A fix-and-optimize approach for the multi-level capacitated lot sizing problem

Stefan Helber, Florian Sahling*

Leibniz Universität Hannover, Institut für Produktionswirtschaft, Königsworther Platz 1, D-30167 Hannover, Germany

ARTICLE INFO

Article history:

Received 25 June 2008

Accepted 22 August 2009

Available online 3 September 2009

Keywords:

Lot sizing

MLCLSP

Lead times

MIP

Fix-and-optimize

Decomposition

ABSTRACT

This paper presents an optimization-based solution approach for the dynamic multi-level capacitated lot sizing problem (MLCLSP) with positive lead times. The key idea is to solve a series of mixed-integer programs in an iterative fix-and-optimize algorithm. Each of these programs is optimized over all real-valued variables, but only a small subset of binary setup variables. The remaining binary setup variables are tentatively fixed to values determined in previous iterations. The resulting algorithm is transparent, flexible, accurate and relatively fast. Its solution quality outperforms those of the approaches by Tempelmeier/Derstroff and by Stadler.

© 2009 Elsevier B.V. All rights reserved.

1. Introduction

This paper treats the problem to determine time-phased production quantities (lot sizes) for multi-level production systems in which a changeover at a resource from one product type to another requires a setup time and/or causes setup cost. The capacity of the resources is limited. The time-phased demand is assumed to be given and has to be satisfied. To date, this type of lot sizing problem cannot be solved satisfactorily within computerized Material Requirements Planning (MRP) modules of Enterprise Resource Planning (ERP) systems. The reason is that these systems often ignore the capacity limits of the production system while computing the lot sizes. This leads to infeasible production schedules which result in long and unpredictable lead times and large in-process inventories.

The problem of finding capacity-feasible production quantities for a multi-stage production system that minimize setup and holding cost has been formally stated

as the multi-level capacitated lot sizing problem (MLCLSP), see Billington et al. (1983). For capacitated production systems with non-zero setup times, the question whether a feasible production plan (without overtime) exists has shown to be already $\mathcal{NP}$ -complete, see Maes et al. (1991). Several authors have therefore developed heuristics for the problem. Multi-level capacitated lot sizing is hence both practically important and scientifically challenging.

From a practical point of view, generic lot sizing approaches for ERP systems should meet several requirements. First, they should be adaptable to specific aspects of a concrete production system. This favors flexible approaches based on mathematical programming as opposed to “hard-wired” procedural heuristics that follow a highly problem-specific logic. Second, they should enable the planner to find feasible solutions of acceptable quality quickly and then let him decide to invest additional computation time to improve the solution. Third, if lot sizing on the one hand and sequencing/scheduling on the other hand are treated separately, it should always be possible to disaggregate the (aggregated) lot sizes into a detailed production schedule in continuous time. In a multi-level production system, this can only be guaranteed if positive lead times between the

* Corresponding author.

E-mail address: sahling@prod.uni-hannover.de (F. Sahling).

production stages are considered. Assume a serial product structure with one end product that requires a single component. Both items are produced on different resources. Furthermore, assume that the resource for the end product is fully loaded to produce a single unit of the end product in a period. In this case, it is not feasible to produce a unit of the component in the same period so that it enters the unit of the end product at the beginning of the same period. Therefore, this unit of the component must be produced at least one period ahead. For this reason, a positive lead time is required.

Reviews of the literature on dynamic lot sizing are given by Bahl et al. (1987), Maes and van Wassenhove (1988), Gupta and Keung (1990), Salomon et al. (1991) and Buschkuhl et al. (2008). Kuik et al. (1994) relate lot sizing to batching and comment on some general criticism of batching analysis. The progress with single-item lot sizing is analyzed by Wolsey (1995) and Brahimi et al. (2006). Drexel and Kimms (1997) discuss models that consider both lot sizing and scheduling. Karimi et al. (2003) give a review of solution approaches for single-stage capacitated lot sizing problems and Jans and Degraeve (2007) focus on metaheuristics for dynamic lot sizing. Quadt and Kuhn (2008) review capacitated lotsizing problems with extensions. Table 1 classifies papers on the MLCLSP by the general solution approach, i.e. mathematical programming-based approaches, Lagrangean relaxation and decomposition, decomposition and aggregation, local search and metaheuristics and finally greedy heuristics. Some of these papers are discussed below in more detail.

In Tempelmeier and Helber (1994), Helber (1995) and Tempelmeier and Derstroff (1996), early product-oriented decomposition approaches for the MLCLSP are presented. The Tempelmeier/Derstroff-heuristic (TDH) is until now

the fastest available method for the MLCLSP. It is based on a Lagrangean relaxation of the MLCLSP which then decomposes into single-product uncapacitated lot sizing problems of the type studied by Wagner and Whitin (1958). The Lagrangean relaxation leads to a lower bound on the objective function value. A heuristic finite scheduling procedure is used to create a feasible solution and to compute an upper bound on the optimal objective function value. While the algorithm is fast, it is difficult to describe and implement as well as inflexible with respect to modifications of the underlying problem. The solution quality especially for large problem instances offers opportunities for improvement. Katok et al. (1998) present a linear-programming (LP)-based approach that works with a heuristic modification of the coefficients of the production quantities in both the objective function and the constraints. Tempelmeier (2008, p. 342) shows that this concept is very vulnerable when setup times occur and capacity limits are tight. In these situations existing feasible solutions (without overtime) are not found.

Stadtler (2003) proposes a mixed-integer programming-based heuristic that solves a series of subproblems through internally rolling schedules with time windows. For the periods within the time window of a particular subproblem, the simple plant location variant of the lot sizing problem is used to speed up the optimization process. Stadtler's approach delivers high-quality solutions for problems with zero lead times, but cannot deal with positive lead times (Stadtler, 2003, p. 501). The reason is that in a general product structure multiple relations with different lead times between end product quantities and time-phased resource requirements can occur. Hence, this makes a consistent disaggregation into a detailed production schedule in continuous time impossible. In addition, solution times for his approach increase substantially as the problem size (number of binary setup variables) increases. A variant of Stadtler's general approach of internally rolling schedules for the Capacitated Lot Sizing Problem with Linked Lot Sizes (Haase, 1994, 1998) is presented by Sürie and Stadtler (2003). It is based on an extended model formulation and valid inequalities to yield a tight formulation that speeds up the branch&bound process.

Belvaux and Wolsey (2001) show how to develop tight formulations for several special problem features occurring in practice. Rossi (2005) develops a time-oriented decomposition similar to the one by Stadtler where some of the setup variables are initially relaxed while others are iteratively fixed. Unfortunately, the author provides no direct comparison to the procedures by Stadtler and by Tempelmeier. Pitakaso et al. (2006) present a decomposition algorithm for the MLCLSP based on a limited subset of products and periods. Each problem in the decomposition is solved to optimality and a capacity reservation mechanism is used to reflect products and periods "outside" of the current problem. The decomposition itself is controlled by an "ant colony optimization" algorithm. The computation times that are necessary to find better results than with Stadtler's method appear to be quite high (20–30 min) and then the average improvement is

Table 1
Literature on the MLCLSP.

<i>Mathematical programming approaches</i>
Billington et al. (1986), Maes et al. (1991), Pochet and Wolsey (1991), Kuik et al. (1993), Clark and Armentano (1995), Harrison and Lewis (1996), Stadtler (1996), Stadtler (1997), Katok et al. (1998), Belvaux and Wolsey (2000, 2001), Rossi (2005), Stadtler (2003), Sürie and Stadtler (2003)
<i>Lagrangean relaxation and decomposition</i>
Tempelmeier and Derstroff (1993, 1996), Özdamar and Barbarosoglu (1999, 2000), Moorkanat (2000), Chen and Chu (2003)
<i>Decomposition and aggregation</i>
Tempelmeier and Helber (1994), Helber (1994), Boctor and Poulin (2005)
<i>Local search and metaheuristics</i>
Salomon et al. (1993), Kuik et al. (1993), Helber (1994, 1995), Barbarosoglu and Özdamar (2000), Hung and Chien (2000), Özdamar and Barbarosoglu (2000), Özdamar and Bozyel (2000), Gutierrez et al. (2001), Xie and Dong (2002), Berretta and Rodrigues (2004), Berretta et al. (2005), Pitakaso et al. (2006)
<i>Greedy heuristics</i>
Clark and Armentano (1995), França et al. (1997)

small. Berretta and Rodrigues (2004) describe a memetic algorithm which is also population-based. However, they compare their results to those presented by Tempelmeier only for tiny problems with 40 binary variables. The picture is similar for a similar algorithm by Berretta et al. (2005): Relatively large deviations from optimal solutions occur already for problems with only 60 binary variables.

In this paper we present a mathematical programming-based algorithm for the MLCLSP that is flexible and transparent that allows to trade in solution time for solution quality and that can deal with positive lead times. In an iterative fix-and-optimize approach (Pochet and Wolsey, 2006, p. 113, call this “Exchange”), a sequence of mixed-integer programs (MIPs) is solved over all real-valued decision variables and a subset of the binary setup variables. The solution with respect to the binary variables is a fixed parameter for the next MIPs that optimize other binary variables. The optimization of the algebraic decision model is done within the MIP solver and therefore the approach is flexible. The user can trade in solution time for solution quality by deciding about the number of binary variables to be treated within a single MIP and about the number of iterations in which these are optimized and fixed again. Even large problem instances from the literature can be solved with a high solution quality within seconds or few minutes. We study both the cases of zero and of positive lead times and compare our results to those for the approaches by Tempelmeier and Derstroff (1996) and Stadtler (2003) in an extensive numerical study. While the method by Tempelmeier and Derstroff (1996) is extremely fast (and much faster than ours), our method yields a substantially higher solution quality. It outperforms the one presented by Stadtler (2003) with respect to both solution quality and computation time.

The remainder of this paper is organized as follows: in Section 2 we give a formal definition of the MLCLSP. The algorithm to solve the problem is presented in Section 3. Numerical results comparing our method to those developed by Tempelmeier and by Stadtler are reported in Section 4. The paper ends with some conclusions and suggestions for further research (Section 5).

2. Problem statement and model formulation

The objective of the MLCLSP is to determine production quantities Q_{kt} and end-of-period inventory levels Y_{kt} of product k in period t so that the sum of setup, holding and overtime cost is minimized. The demand d_{kt} per product and period at each production stage is given and has to be satisfied. Whenever a product is produced during a period, a setup is required during this period which results in both setup cost and setup time. The setup state $\gamma_{kt} \in \{0, 1\}$ for product k in period t is assumed to be lost at the end of this period. The regular capacity b_{jt} of a resource j in a period t can be extended by using overtime O_{jt} . We now formally state the MLCLSP using the notation in Table 2, see Billington et al. (1983) and also Stadtler (2003).

Table 2
Notation for the MLCLSP.

<i>Sets</i>	
$k, i \in \mathcal{K}$	Products
$t \in \mathcal{T}$	Periods
$j \in \mathcal{J}$	Resources
$\mathcal{K}_j$	Set of products requiring resource j
$\mathcal{N}_k$	Set of immediate successors of product k
<i>Parameters</i>	
a_{ki}	Number of units of product k required to produce one unit of product i
b_{jt}	Available capacity of resource j in period t
B	Big number
d_{kt}	External demand of product k in period t
h_k	Holding cost of product k per unit and period
oc_{jt}	Overtime cost per unit of overtime at resource j in period t
s_k	Setup cost of product k
tp_k	Production time per unit of product k
ts_k	Setup time of product k
z_k	Planned lead time of product k
<i>Decision variables</i>	
O_{jt}	Overtime at resource j in period t
Q_{kt}	Production quantity (lot size) of product k in period t
Y_{kt}	Planned end-of-period inventory of product k in period t
γ_{kt}	Binary setup variable of product k in period t

Model MLCLSP:

$$\text{Minimize } Z = \sum_{k \in \mathcal{K}} \sum_{t \in \mathcal{T}} (s_k \cdot \gamma_{kt} + h_k \cdot Y_{kt}) + \sum_{j \in \mathcal{J}} \sum_{t \in \mathcal{T}} oc_{jt} \cdot O_{jt} \quad (1)$$

subject to:

$$Y_{k,t-1} + Q_{k,t-z_k} - \sum_{i \in \mathcal{N}_k} a_{ki} \cdot Q_{it} - Y_{kt} = d_{kt} \quad \forall k, t \quad (2)$$

$$\sum_{k \in \mathcal{K}_j} (tp_k \cdot Q_{kt} + ts_k \cdot \gamma_{kt}) \leq b_{jt} + O_{jt} \quad \forall j, t \quad (3)$$

$$Q_{kt} - B \cdot \gamma_{kt} \leq 0 \quad \forall k, t \quad (4)$$

$$Q_{kt}, Y_{kt} \geq 0 \quad \forall k, t \quad (5)$$

$$O_{jt} \geq 0 \quad \forall j, t \quad (6)$$

$$\gamma_{kt} \in \{0, 1\} \quad \forall k, t \quad (7)$$

$$Y_{k0} = Y_{kT} = 0 \quad \forall k \quad (8)$$

The objective function (1) states that the sum of the setup, holding and overtime cost has to be minimized. The inventory balance equations (2) reflect the multi-level production structure. Note that due to the lead time z_k , a production quantity of product k in period $t - z_k$ is available in period t to satisfy the external demand d_{kt} or to be used in the production of a succeeding product i . If we set $z_k = 1, \forall k$, any feasible lot sizing decision can be transformed into a feasible schedule, see Section 1.

Inequalities (3) state that production quantities and setups must meet the capacity constraints for all the resources. Inequalities (4) ensure that a machine is set up

for product k in period t , if the product is produced in this period. Production quantities and inventory levels (5) as well as planned overtime (6) cannot be negative. Setup variables are binary (7) and there are neither initial nor ending inventories (8). Note that this version of the MLCLSP is always formally feasible as there is no limit on overtime. The undesired use of overtime, however, is priced out of the solution by the algorithm presented in the next section.

3. An iterative optimization-based heuristic

3.1. Basic idea of the fix-and-optimize heuristic

The standard formulation of the MLCLSP based on production and inventory quantities as stated in Section 2 typically leads for all but tiny problem instances to solution times of a MIP solver that are prohibitively large. The number of binary setup variables ($|K| \cdot |T|$) determines most of the numerical effort, while the number of real-valued variables is of secondary importance. The basic idea of our proposal is therefore to solve in an iterative fashion a series of subproblems that are derived from the MLCLSP. In each iteration, most of the binary setup variables γ_{kt} are set to a fixed value $\bar{\gamma}_{kt}^{fix}$. This reduces the number of “free” binary variables in the subproblem of the current iteration. The resulting subproblems are then solved by a MIP solver to optimality. As the number of “free” binary variables of the subproblem is much smaller than in the original MLCLSP, the solution time for a subproblem is very small. This yields a new temporary solution for the setup variables of the current subproblem. At least some of them are fixed in the next iteration when a different subset of binary variables is optimized. In each subproblem the complete set of real-valued decision variables over all products, periods and machines is considered. Thus, it is not necessary to freeze parts of the plan nor to deal with end-of-horizon effects like those encountered by Stadtler in his internally rolling schedules with lot sizing windows.

In a Relax-and-Fix heuristic (Pochet and Wolsey, 2006, p. 109) like the ones proposed by Stadtler (2003) or Federgruen et al. (2007), the binary setup variables of the original MLCLSP are divided into three groups for each subproblem: the first group contains those that are fixed, the second those that are optimized and the third contains those for which the integrality constraints are relaxed. During a Relax-and-Fix heuristic a portion of the binary setup variables is considered relaxed. This leads to an underestimated capacity consumption due to setup times. For this reason, reformulations of the MLCLSP, e.g. the simple plant location (SPL) formulation (see Stadtler, 2003), are used to strengthen the lower bounds. As the number of additional variables increases in the SPL formulation the computing times increase as well. This is not required for the Fix-and-Optimize heuristic as it operates only with the first two groups. The major advantage of our Fix-and-Optimize heuristic is, hence, that in each iteration a formally feasible solution is obtained.

Table 3

Additional notation for the MLCLSP-SUB.

Sets	
$(k, t) \in \mathcal{KT}$	Set of product-period combinations
$\mathcal{KT}^{opt} \subseteq \mathcal{KT}$	Set of product-period combinations for which binary variables γ_{kt} are optimized in the current subproblem
$\mathcal{KT}^{fix} \subseteq \mathcal{KT}$	Set of product-period combinations for which binary variables γ_{kt} are fixed in the current subproblem
Parameters	
$\bar{\gamma}_{kt}$	Exogenous value of the fixed setup variables γ_{kt} in the current subproblem

3.2. Model formulation for the subproblem

With the additional notation in Table 3, the subproblem for a given set $\mathcal{KT}^{fix} \subseteq \mathcal{KT}$ of fixed setup variables can be stated as follows:

Model MLCLSP-SUB:

Minimize objective function (1) subject to constraints (2)–(8) and the additional constraints

$$\gamma_{kt} = \bar{\gamma}_{kt} \quad \forall (k, t) \in \mathcal{KT}^{fix} \quad (9)$$

These additional constraints suffice to limit the optimization of the binary setup variables to the set $\mathcal{KT}^{opt} = \mathcal{KT} \setminus \mathcal{KT}^{fix}$. (It is possible to state the problem mathematically in a more compact form, but modern solvers detect and resolve the redundancies of the current formulation automatically.)

3.3. Definition of subsets of binary setup variables

The bigger the relative number $|\mathcal{KT}^{opt}|/|\mathcal{KT}|$ of free binary variables in each subproblem MLCLSP-SUB, the more time-consuming is the solution of the resulting MIP and the higher is the quality of the solution that can be found. The different product-period combinations $\mathcal{KT}^{opt}$ over which the setup pattern is optimized must be closely related in order to offer trade-offs to be used within the optimization. During the development of our algorithm, we experimented with several different ways to define subsets $\mathcal{KT}_s^{opt}$ of binary variables for ordered sets $s \in \mathcal{S}$ of subproblems, out of which three turned out to be particularly useful:

- *Product-oriented decomposition:* Each subproblem s corresponds to a (single) product k . In each subproblem, all periods t are treated. That is, setup decisions are optimized for single products over the complete planning horizon.
- *Resource-oriented decomposition:* Each subproblem s corresponds to a (single) resource j and a subset of periods t . The subset of periods contains four successive periods. Two successive subproblems s related to the same resource j have an overlap of two periods, e.g. periods 1–4 for the first subset, periods 3–6 for the second, etc. In each such subproblem, all products $k \in \mathcal{K}_j$ requiring resource j are considered.

- *Process-oriented decomposition*: Each subproblem s corresponds to a subset of periods t and a direct predecessor–successor-relationship between a product k and one of its immediate successors $i \in \mathcal{N}_k$. For each direct predecessor–successor-relationship two subproblems s are defined. The first one covers the first half of the planning horizon and the other one the second half.

Each of these decompositions reflects a particular perspective on the problem. We combine these perspectives in the following four variants of our algorithm.

- *Variant 1*: Product-oriented decomposition only;
- *Variant 2*: Product-oriented decomposition first, then resource-oriented decomposition;
- *Variant 3*: Product-oriented decomposition first, then process-oriented decomposition;
- *Variant 4*: Product-oriented decomposition first, then resource-oriented decomposition, finally process-oriented decomposition.

Each variant can either be treated just once or repeated until a local optimum is reached.

In each of the four variants we start with a product-oriented decomposition. We observed that the sequence in which different products are treated during the early steps of the algorithm is important. For this reason, we try to start with those products that “cause” most of the cost. If multiple product types are produced on a machine in a period in which overtime cost occur, it is not possible to allocate the overtime cost to the product types based on an unambiguous cause-and-effect reasoning. We therefore allocate overtime cost proportional to the time consumption of the product types. In spite of this problem, we attempt to estimate product-specific cost Z_k from the solution of the LP-relaxation of problem MLCLSP (1)–(8). This LP-relaxation yields an objective function value Z^{rel} . From the solution of this LP-relaxation we determined cost values Z_k with

$$Z^{rel} = \sum_{k \in \mathcal{K}} Z_k \quad (10)$$

for each product type as follows:

$$Z_k = \sum_{t \in T} (s_k \cdot \gamma_{kt}^{rel} + h_k \cdot Y_{kt}) + \frac{\sum_{t \in T} (tp_k \cdot Q_{kt} + ts_k \cdot \gamma_{kt}^{rel}) (\sum_{t \in T} OC_{j(k),t} O_{j(k),t})}{\sum_{k \in \mathcal{K}_{(j)}} \sum_{t \in T} (tp_k \cdot Q_{kt} + ts_k \cdot \gamma_{kt}^{rel})} \quad (11)$$

based on the LP-relaxation γ_{kt}^{rel} of the binary setup variables γ_{kt} . Here, the cost of overtime is charged to the products proportionally to the capacity usage of the respective resource. In the product-oriented decomposition, products are ordered according to decreasing cost Z_k to determine the ordered set $\mathcal{S}$ of subproblems. (We also tested other sequences (i.e. other ordered sets of subproblems), but in general the cost-based sequence performed best.)

To tighten the “Big-B” constraints (4) in this LP-relaxation, we computed the time-phased echelon

demand D_{kt} recursively

$$D_{kt} = d_{k,t+z_k} + \sum_{i \in \mathcal{N}_k} a_{ki} \cdot D_{i,t+z_k} \quad (12)$$

to determine a parameter M_{kt}

$$M_{kt} = \sum_{\tau \in T, \tau \geq t} D_{k\tau} \quad (13)$$

which describes the maximum quantity that can possibly be produced of product k in period t , if there is no backlog in the production plan. This parameter M_{kt} is used to substitute the parameter B in constraint (4)

$$Q_{kt} - M_{kt} \cdot \gamma_{kt} \leq 0 \quad \forall k, t. \quad (14)$$

This leads to larger values γ_{kt}^{rel} and hence a tighter LP-relaxation than one would get for an arbitrarily large value of the parameter B .

Furthermore, we considered a product order related to the product specific costs provided by the LP-relaxation of the SPL formulation. Although the LP-relaxation of the SPL formulation provides tighter bounds than those obtained by the standard formulation, the product order provided by the standard formulation leads to significantly better results.

The first subproblem of the resource-oriented decomposition is related to the first resource ($j=1$). The resources are considered in ascending order. The same holds for the process-oriented decomposition. Here, the products and the respective immediate successors are considered in ascending order as well, i.e. neither for the resource- nor for the process-oriented decomposition a specific order of the subproblems is determined. The reason is that after a first round of the product-oriented decomposition usually a quite reasonable setup pattern has already been found. In the numerical tests, it was our impression that from that point on the sequence of the subproblems in the resource- and process-oriented decomposition is less relevant.

3.4. Iterative algorithm

The basic structure of our algorithm is outlined in Algorithm 1. Initially, a setup is planned for each product in each period and the cost of this solution is determined. (We found out that from this starting point the algorithm can price out economically unattractive setup decisions most quickly.) We then go through the ordered set of subproblems of the respective variant of our algorithm (1–4, see above) either once ($l^{max} = 1$) or until we reach a local optimum ($l^{max} = \infty$). In our algorithm, each subproblem s is solved to optimality and the solution yields an objective function value Z that is at least as good as the currently best value Z^{old} . In each iteration l a new solution is only accepted if it yields lower cost than the current plan. As in the approach proposed by Stadler (2003, p. 495) a capacity infeasible solution (with overtime) is never considered as a candidate for a best solution, if there is already a known capacity feasible solution. The Boolean variable *CapFeas* indicates whether a capacity feasible solution has already been found.

Algorithm 1. Two-phase algorithm

```

 $\bar{\gamma}_{kt} = 1, \forall (k, t) \in \mathcal{KT}$ 
 $\mathcal{KT}^{fix} \leftarrow \mathcal{KT}$ 
solve MLCLSP-SUB and determine objective function value  $Z$ 
 $Z^{new} = Z$ 
if  $\sum_{j \in \mathcal{J}} \sum_{t \in \mathcal{T}} O_{jt} = 0$  then
   $CapFeas = \text{yes}$ 
else
   $CapFeas = \text{no}$ 
end if
 $l = 0$ 
repeat
   $l = l + 1$ 
   $Z^{old} = Z^{new}$ 
  for each decomposition  $\mathcal{S}$  in the current variant of the algorithm do
    for each subproblem  $s \in \mathcal{S}$  do
      determine  $\mathcal{KT}_s^{opt}$  and  $\mathcal{KT}_s^{fix} = \mathcal{KT} \setminus \mathcal{KT}_s^{opt}$  for subproblem  $s$ 
      solve MLCLSP-SUB and determine objective function value  $Z$ 
      if  $\sum_{j \in \mathcal{J}} \sum_{t \in \mathcal{T}} O_{jt} = 0$  then
         $CapFeas^{new} = \text{yes}$ 
      else
         $CapFeas^{new} = \text{no}$ 
      end if
      if  $Z < Z^{old}$  and ( $CapFeas^{new}$  or (not( $CapFeas^{new}$ ) and not( $CapFeas$ ))) then
         $\bar{\gamma}_{kt} = \gamma_{kt}, \forall (k, t) \in \mathcal{KT}_s^{opt}$ 
         $Z^{new} = Z$ 
        if  $CapFeas^{new}$  then
           $CapFeas = \text{yes}$ 
        end if
      end if
    end for
  end for
until  $l = l^{max}$  or  $Z^{new} \geq Z^{old}$ 

```

Our algorithm was implemented in Delphi and we used the CPLEX 10.2 callable library to solve the MIPs on a 2.13 GHz Intel Pentium Core2 machine with 4 GB of RAM.

4. Numerical results**4.1. Test sets and reference values**

In order to evaluate the performance of our approach, we used and extended test instances without lead times that were developed and documented in detail by [Stadtler and Sürie \(2000\)](#) and used in [Stadtler \(2003\)](#). Out of their test sets, we used test sets A⁺, B⁺, C, D and E as described in [Table 4](#). These test sets cover different product structures with 10–100 items and between 16 and 24 periods.

In the original test sets by [Stadtler and Sürie \(2000\)](#), lead times are not considered. However, as modeling lead times is a practical necessity in multi-level production systems, we tested our approach for problems with and without lead times. In the latter case, we assumed a lead time of one period for each product except for end products. This increases the number of necessary periods by $u - 1$ periods for product structures with u production stages to allow for all production steps at all intermediate production stages. We therefore added the necessary number of periods and shifted the demand data by the number of additional periods into the future. (The product structures in Problem Class A⁺, for example, have $u = 3$ production stages and 24 demand periods. To consider lead times, we therefore work with $24 + (u - 1) = 24 + 2 = 26$ periods such that the time-shifted demand occurs in periods 3–26.) The last column of [Table 4](#) presents triple values in the form “a/b/c”. The first entry “a” indicates the total number of instances in this problem class, whereas entries “b” and “c” give the number of instances in this class for which a capacity-feasible solution (without overtime) is known for cases with zero lead times and lead times of one period, respectively. For cases with zero lead times, [Stadtler and Sürie \(2000\)](#) report lower bounds on the objective function value (computed via the simple plant location (SPL) formulation of the MLCLSP) and upper bounds on the best solutions known to them. For cases with lead times of one period, we computed the respective bounds ourselves. To this end, we also used the SPL formulation of the MLCLSP ([Stadtler, 2003](#)) in a truncated branch&bound approach of the CPLEX MIP solver. For each problem instance of Problem Classes A⁺ and B⁺, we allowed for a maximum computation time of 1.5 h of CPU time. The respective limit was 3 h for Problem Classes C and D and 6 h for Problem Class E. In our numerical study, we only tested the different algorithms on those subsets of the test instances for which solutions without overtime are known.

We compare our results to those obtained by the Lagrangean decomposition method developed by [Tempelmeier and Derstroff \(1996\)](#) and the time decomposition approach by [Stadtler \(2003\)](#). Both authors provided us with an implementation of their respective method to allow for a fair comparison. For problem instances without lead times, we were able to compare our results to both competing approaches. For the instances with lead times, our approach can only be compared to the one by [Tempelmeier and Derstroff \(1996\)](#) as Stadtler's time decomposition cannot deal with lead times. It should also be noted that the

Table 4
Selected test sets.

	Products	Demand periods	Resources	Setup times	Test instances
Class A ⁺	10	24	3	No	120/108/116
Class B ⁺	10	24	3	Yes	312/312/312
Class C	40	16	6	No	180/132/153
Class D	40	16	6	Yes	80/72/74
Class E	100	16	10	No	150/150/150

implementation of the Tempelmeier–Derstroff-Heuristic (TDH) (courtesy of Tempelmeier's group) occasionally led to solutions with some (very small) overtime use. For this reason, we decided to take the setup pattern from the TDH solution and to re-compute the production plan by solving the remaining linear program with real-valued decision variables, given this setup pattern. We found that this additional step led to production quantities that were indeed capacity feasible (without overtime) and report the objective function values for these slightly better solutions of a “polished version” of the TDH.

We found that the method proposed by Stadtler is quite time-consuming. The implementation of the algorithm (courtesy of Stadtler's group) in XPRESSMP (rel. 17) has an option to impose a limit on the computation time. However, this limit takes effect only if a first feasible solution has been found. For each problem class we determined the maximum computation time of each

combination of problem instance and algorithmic variant of our method. We imposed this maximum value as a time limit for Stadtler's method. In several cases Stadtler's method required more than this time limit to find a first feasible solution. Stadtler's method operates with moving time windows. Based on the parameter settings that he reported to be quite powerful, we used his algorithm with a time window of four successive periods (none of them with a relaxed integrality constraint). In the next iteration, we moved this time window one period into the future. In Stadtler's terms, we worked with a “4/0/1” parameter setting.

Depending on the variant of our algorithm as described in Section 3, in each instance of Problem Classes A^+ and B^+ , we optimized over 3–10% of the binary setup variables, i.e. $0.03 \leq |\mathcal{KT}^{opt}/\mathcal{KT}| \leq 0.1$ in any one instance of Problem MLCLSP-SUB. For the larger problem instances with more products, this fraction of optimized binary variables was

Table 5

Results for class A^+ and class B^+ without lead times.

	Problem class A^+				Problem class B^+			
	ADUB (%)	ADLB (%)	Feas (%)	Time (s)	ADUB (%)	ADLB (%)	Feas (%)	Time (s)
TDH	13.19	37.95	100.00	0.08	12.52	37.28	100.00	0.08
StaH	4.14	26.31	99.07	13.49 ^a	4.11	24.18	84.62	13.50 ^b
<i>Single iteration ($I^{max} = 1$)</i>								
Var 1	4.09	26.50	100.00	1.16	3.82	26.42	100.00	1.56
Var 2	1.80	23.79	100.00	2.72	1.52	23.55	100.00	3.32
Var 3	2.13	24.20	100.00	3.03	1.96	24.13	100.00	3.39
Var 4	1.13	23.00	100.00	3.80	0.84	22.71	100.00	4.72
<i>Multiple iterations ($I^{max} = \infty$)</i>								
Var 1	2.51	24.66	100.00	1.71	2.20	24.40	100.00	2.08
Var 2	1.03	22.87	100.00	4.42	0.62	22.43	100.00	5.24
Var 3	1.67	23.65	100.00	3.75	1.30	23.31	100.00	5.13
Var 4	0.75	22.54	100.00	5.18	0.35	22.12	100.00	7.03

^a Maximum allowed solution time: 14 s.

^b Maximum allowed solution time: 22 s.

Table 6

Results for class C and class D without lead times.

	Problem class C				Problem class D			
	ADUB (%)	ADLB (%)	Feas (%)	Time (s)	ADUB (%)	ADLB (%)	Feas (%)	Time (s)
TDH	6.76	23.07	100.00	0.43	5.68	14.80	100.00	0.28
StaH	2.14	18.33	99.24	279.21 ^a	6.99	14.96	88.89	76.73 ^b
<i>Single iteration ($I^{max} = 1$)</i>								
Var 1	8.32	25.66	100.00	3.83	10.34	19.78	100.00	3.59
Var 2	3.80	20.44	100.00	13.80	4.96	13.89	100.00	7.48
Var 3	4.81	21.65	100.00	16.72	5.33	14.33	100.00	11.45
Var 4	2.11	18.51	100.00	18.96	3.34	12.13	100.00	13.72
<i>Multiple iterations ($I^{max} = \infty$)</i>								
Var 1	5.14	22.03	100.00	7.29	5.28	14.27	100.00	6.20
Var 2	1.57	17.85	100.00	26.40	2.72	11.45	100.00	10.96
Var 3	3.71	20.40	100.00	33.88	4.36	13.26	100.00	19.72
Var 4	1.12	17.36	100.00	29.87	2.15	10.85	100.00	22.25

^a Maximum allowed solution time: 263 s.

^b Maximum allowed solution time: 81 s.

Table 7
Results for class E.

	Without lead times				With lead times			
	ADUB (%)	ADLB (%)	Feas (%)	Time (s)	ADUB (%)	ADLB (%)	Feas (%)	Time (s)
TDH	6.13	17.60	100.00	0.69	7.12	16.26	100.00	1.13
StaH	4.66	15.84	100.00	420.95 ^a	–	–	–	–
<i>Single iteration ($I^{\max} = 1$)</i>								
Var 1	11.01	23.14	100.00	17.68	9.38	18.64	100.00	21.17
Var 2	7.27	18.97	100.00	36.15	5.36	14.27	100.00	42.88
Var 3	6.76	18.45	100.00	49.79	4.65	13.54	100.00	60.83
Var 4	4.38	15.62	100.00	67.34	3.20	11.89	100.00	97.56
<i>Multiple iterations ($I^{\max} = \infty$)</i>								
Var 1	6.71	18.41	100.00	24.48	4.28	13.13	100.00	31.39
Var 2	3.93	15.09	100.00	49.51	3.27	11.95	100.00	62.04
Var 3	4.81	16.24	100.00	68.30	2.99	11.70	100.00	83.45
Var 4	3.12	14.20	100.00	89.47	2.41	11.02	100.00	123.38

^a Maximum allowed solution time: 397 s.

smaller. When dealing with Problem Classes C and D, only 1–7.5% of the binary variables were optimized in a single instance of Problem MLCLSP-SUB. For Problem Class E, this range was 0.5–4% of the setup variables. In each subproblem, we tried to solve as many binary variables to optimality as possible in reasonable time. As a rule of thumb for our current soft- and hardware configuration, the absolute number of binary variables should not exceed 70 variables.

4.2. Results for problem instances without lead times

The results for problem instances without lead times are presented in Tables 5–7. We report for the approach by Tempelmeier (TDH), by Stadtler (StaH) and the four variants of our algorithm the following values: “ADUB” denotes the average relative deviation of the solution from the best known upper bound as reported by Stadtler and Sürie. “ADLB” denotes the average relative deviation of the respective heuristic solution from the lower bound as reported by Stadtler and Sürie. “Feas” is the fraction of problem instances that could be solved without the use of overtime (which is extremely costly in all the instances) and “Time” is the computation time.

The results show the following: the heuristic by Tempelmeier is so fast (compared to all other approaches) that its computation time can in fact be neglected. For problems without lead times, however, it yields the worst solution quality if compared to the approach by Stadtler and to most variants of our method. If one allows for multiple iterations of Variants 2–4 of our method, it outperforms Stadtler's method both with respect to the computation time and to the quality of the solution, even though the improvement of the solution quality is limited. It is also interesting to note that on average a single iteration of Variant 4 of our algorithm yields better solutions than multiple iterations of Variant 1 of the method. This indicates that changing the perspective in the process of optimizing the setup pattern is a worthwhile undertaking. The results also indicate that it is in general useful to go through multiple iterations of any

variant of the algorithm until a local optimum is found. The percentage increase of the solution time appears to be no more than about 50% of the time for a single iteration. Note that for Problem Classes C and E, the average time required for Stadtler's method to find a capacity feasible solution (without overtime) extended the time limit derived from the maximum solution time of our method as explained in Section 4.1.

Furthermore, we tested a hybrid version of our Fix-and-Optimize heuristic using the setup pattern provided by the Tempelmeier/Derstroff heuristic as a starting point. We found that this hybrid method leads the Fix-and-Optimize algorithm to good setup patterns quickly and can hence reduce the computational effort to some extent. However, the (in our eyes) highly valuable flexibility of our solution approach gets lost as the TDH can only be used to solve the model formulation of the MLCLSP considered by Tempelmeier and Derstroff. It is not possible to determine the initial setup pattern via the TDH if additional (simple) constraints, e.g. minimum lot sizes or safety stocks, are considered. In this case, the quite elaborate TDH would have to be adjusted significantly while only small changes of the model formulation are necessary for the Fix-and-Optimize heuristic.

4.3. Results for problem instances with lead times

In Tables 8, 9 and the right hand part of Table 7 we present the results for lead times of one period. (Remember that for these instances Stadtler's method cannot be applied.) The results look very similar to those for problem instances without lead times. If one has to deal with lead times, our method is the one that currently delivers the highest solution quality.

5. Conclusions and outlook

We have presented a mathematical-programming-based approach to solve the MLCLSP with lead times. Modeling lead times is necessary, if one wants to be able to disaggregate the time-phased lot sizes into a detailed

Table 8Results for class A⁺ and class B⁺ with lead times.

	Problem class A ⁺				Problem class B ⁺			
	ADUB (%)	ADLB (%)	Feas (%)	Time (s)	ADUB (%)	ADLB (%)	Feas (%)	Time (s)
TDH	12.92	34.71	100.00	0.10	11.38	30.82	100.00	0.09
<i>Single iteration ($I^{\max} = 1$)</i>								
Var 1	3.94	23.66	100.00	1.38	4.16	22.11	100.00	1.78
Var 2	1.66	20.99	100.00	2.81	1.50	18.98	100.00	3.67
Var 3	2.09	21.51	100.00	2.95	1.71	19.25	100.00	3.65
Var 4	0.76	19.93	100.00	5.02	0.35	17.61	100.00	5.90
<i>Multiple iterations ($I^{\max} = \infty$)</i>								
Var 1	2.44	21.93	100.00	1.80	2.11	19.71	100.00	2.69
Var 2	0.87	20.06	100.00	4.65	0.42	17.70	100.00	4.66
Var 3	1.52	20.86	100.00	3.56	1.08	18.50	100.00	5.07
Var 4	0.53	19.65	100.00	5.86	0.07	17.29	100.00	6.85

Table 9

Results for class C and class D with lead times.

	Problem class C				Problem class D			
	ADUB (%)	ADLB (%)	Feas (%)	Time (s)	ADUB (%)	ADLB (%)	Feas (%)	Time (s)
TDH	9.00	24.20	100.00	0.50	7.30	15.64	100.00	0.30
<i>Single iteration ($I^{\max} = 1$)</i>								
Var 1	8.55	23.65	100.00	3.97	10.04	18.53	97.30	3.65
Var 2	3.63	18.09	100.00	10.48	4.97	13.04	97.30	7.82
Var 3	4.63	19.29	100.00	12.20	5.07	13.17	97.30	10.74
Var 4	1.56	15.71	100.00	21.77	2.69	10.59	97.30	17.74
<i>Multiple iterations ($I^{\max} = \infty$)</i>								
Var 1	4.64	19.30	100.00	5.97	4.97	13.06	97.30	5.01
Var 2	1.54	15.68	100.00	17.23	2.74	10.63	97.30	11.63
Var 3	3.58	18.09	100.00	21.21	4.11	12.13	97.30	15.61
Var 4	1.19	15.29	100.00	27.69	2.07	9.91	97.30	22.07

production schedule in continuous time. Our method is based on an iterative optimization of a series of subproblems. Each of them includes all the real-valued decision variables, but only a specific limited set of “free” binary variables. The approach is easy to describe and to implement and delivers high-quality solutions. In our view, the most important aspect of our method is its flexibility: it is straightforward to change or add constraints such as maximum inventory levels, minimum lot sizes, parallel machines, etc. to the standard formulation of the MLCLSP. These changes of the model would require a substantial re-design of the complex decomposition method by Tempelmeier and Derstroff (1996), the only other method so far that can efficiently solve larger instances of the MLCLSP with positive lead times. In addition, our method is quite easy to modify by defining different subsets and sequences of subproblems to be treated in the different phases of the algorithm. By defining larger subproblems (with a larger (absolute or relative) number of binary variables), one can trade in solution time to get more solution quality and vice versa. However, the results show that it may be sufficient to treat less than 10% of the binary setup variables in any one

instance of the problem MLCLSP-SUB and still find high quality solutions, similar to two-opt and three-opt approaches for the Traveling Salesman Problem. We consider the flexibility with respect to both the lot sizing model and the algorithm to be an important aspect of our work. Further research should address other dynamic multi-level lot sizing problems like those with linked lot sizes where the setup state of a resource can be carried over to a subsequent period.

References

- Bahl, H.C., Ritzman, L.P., Gupta, J.N.D., 1987. Determining lot sizes and resource requirements: a review. *Operations Research* 35, 329–345.
- Barbarosoglu, G., Özdamar, L., 2000. Analysis of solution space-dependent performance of simulated annealing: the case of the multi-level capacitated lot sizing problem. *Computers and Operations Research* 27, 895–903.
- Belvaux, G., Wolsey, L.A., 2000. bc-prod: a specialized branch-and-cut system for lot-sizing problems. *Management Science* 46, 724–738.
- Belvaux, G., Wolsey, L.A., 2001. Modelling practical lot-sizing problems as mixed-integer programs. *Management Science* 47, 993–1007.
- Berretta, R., França, P.M., Armentano, V.A., 2005. Metaheuristic approaches for the multilevel resource-constrained lot-sizing problem with setup and lead times. *Asia-Pacific Journal of Operational Research* 22, 261–286.

- Berretta, R., Rodrigues, L.F., 2004. A memetic algorithm for a multistage capacitated lot-sizing problem. *International Journal of Production Economics* 87, 67–81.
- Billington, P.J., McClain, J.O., Thomas, L.J., 1983. Mathematical programming approaches to capacity-constrained MRP systems: review, formulation and problem reduction. *Management Science* 39, 1126–1141.
- Billington, P.J., McClain, J.O., Thomas, L.J., 1986. Heuristics for multilevel lot-sizing with a bottleneck. *Management Science* 32, 989–1006.
- Bector, F.F., Poulin, P., 2005. Heuristics for the n -product, m -stage, economic lot sizing and scheduling problem with dynamic demand. *International Journal of Production Research* 43, 2809–2828.
- Brahimi, N., Dauzère-Pérès, S., Najid, N.M., Nordli, A., 2006. Single item lot sizing problems. *European Journal of Operational Research* 168, 1–16.
- Buschkühl, L., Sahling, F., Helber, S., Tempelmeier, H., 2008. Dynamic capacitated lot-sizing problems — a classification and review of solution approaches. *Operations Research Spectrum*, 1–31. DOI: 10.1007/s00291-008-0150-7.
- Chen, H., Chu, C., 2003. A Lagrangian relaxation approach for supply chain planning with order/setup costs and capacity constraints. *Journal of Systems Science and Systems Engineering* 12, 98–110.
- Clark, A.R., Armentano, V.A., 1995. A heuristic for a resource-capacitated multi-stage lot-sizing problem with lead times. *Journal of the Operational Research Society* 46, 1208–1222.
- Drexel, A., Kimms, A., 1997. Lot sizing and scheduling — survey and extensions. *European Journal of Operational Research* 99, 221–235.
- Federgruen, A., Meissner, J., Tzur, M., 2007. Progressive interval heuristics for multi-item capacitated lot-sizing problems. *Operations Research* 55, 490–502.
- França, P.M., Armentano, V.A., Berretta, R.E., Clark, A.R., 1997. A heuristic method for lot-sizing in multi-stage systems. *Computers and Operations Research* 24, 861–874.
- Gupta, Y.P., Keung, Y., 1990. A review of multi-stage lot-sizing models. *International Journal of Operations and Production Management* 10, 57–73.
- Gutierrez, E., Hernández, W., Süer, G.A., 2001. Genetic algorithms in capacitated lot sizing decisions. In: *Proceedings of the Computing Research Conference 2001*, Mayagüez, Puerto Rico, pp. 1–4.
- Haase, K., 1994. Lot sizing and Scheduling for Production Planning. In: *Lecture Notes in Economics and Mathematical Systems*, vol. 408. Springer, Berlin.
- Haase, K., 1998. Capacitated lot-sizing with linked production quantities of adjacent periods. In: Drexel, A., Kimms, A. (Eds.), *Beyond Manufacturing Resource Planning (MRP II): Advanced Models and Methods for Production Planning*. Springer, Berlin, pp. 127–146.
- Harrison, T.P., Lewis, H.S., 1996. Lot sizing in serial assembly systems with multiple constrained resources. *Management Science* 42, 19–36.
- Helber, S., 1994. Kapazitätsorientierte Losgrößenplanung in PPS-Systemen. Metzler & Poeschel, Verlag für Wissenschaft und Forschung, Stuttgart.
- Helber, S., 1995. Lot sizing in capacitated production planning and control systems. *Operations Research Spektrum* 17, 5–18.
- Hung, Y.-F., Chien, K.L., 2000. A multi-class multi-level capacitated lot sizing model. *Journal of the Operational Research Society* 51, 1309–1318.
- Jans, R., Degraeve, Z., 2007. Meta-heuristics for dynamic lot sizing: a review and comparison of solution approaches. *European Journal of Operational Research* 177, 1855–1875.
- Karimi, B., Ghomi, S.M.T.F., Wilson, J.M., 2003. The capacitated lot sizing problem: a review of models and algorithms. *Omega* 31, 365–378.
- Katok, E., Lewis, H.S., Harrison, T.P., 1998. Lot sizing in general assembly systems with setup costs, setup times, and multiple constrained resources. *Management Science* 44, 859–877.
- Kuik, R., Salomon, M., van Wassenhove, L.N., 1994. Batching decisions: structure and models. *European Journal of Operational Research* 75, 243–263.
- Kuik, R., Salomon, M., van Wassenhove, L.N., Maes, J., 1993. Linear programming simulated annealing and tabu search heuristics for lot sizing in bottleneck assembly systems. *IIE Transactions* 25, 62–72.
- Maes, J., McClain, J.O., van Wassenhove, L.N., 1991. Multilevel capacitated lot sizing complexity and LP-based heuristics. *European Journal of Operational Research* 53, 131–148.
- Maes, J., van Wassenhove, L., 1988. Multi-item single-level capacitated dynamic lot-sizing heuristics: a general review. *Journal of the Operational Research Society* 39, 991–1004.
- Moorkanat, J., 2000. Studies in certain resource loading, scheduling and production control problems in multi-stage production control problems in multi-stage production inventory systems. Ph.D. Thesis, Indian Institute of Technology, Bombay.
- Özdamar, L., Barbarosoglu, G., 1999. Hybrid heuristics for the multi-stage capacitated lot sizing and loading problem. *Journal of the Operational Research Society* 50, 810–825.
- Özdamar, L., Barbarosoglu, G., 2000. An integrated Lagrangean relaxation-simulated annealing approach to the multi-level multi-item capacitated lot sizing problem. *International Journal of Production Economics* 68, 319–331.
- Özdamar, L., Bozyel, M.A., 2000. The capacitated lot sizing problem with overtime decisions and setup times. *IIE Transactions* 32, 1043–1057.
- Pitakaso, R., Almeder, C., Doerner, K.F., Hartl, R.F., 2006. Combining population-based and exact methods for multi-level capacitated lot-sizing problems. *International Journal of Production Research* 44, 4755–4771.
- Pochet, Y., Wolsey, L.A., 1991. Solving multi-item lot-sizing problems using strong cutting planes. *Management Science* 37, 53–67.
- Pochet, Y., Wolsey, L.A., 2006. *Production Planning by Mixed Integer Programming*. Springer, New York, NY.
- Quadt, D., Kuhn, H., 2008. Capacitated lot-sizing with extensions: a review. *4OR: A Quarterly Journal of Operations Research* 6, 61–83.
- Rossi, H., 2005. Ein heuristisches Dekompositionsverfahren für mehrstufige Losgrößenprobleme. Ph.D. thesis, Fachbereich Wirtschaftswissenschaft, Freie Universität Berlin. <http://www.diss.fu-berlin.de/diss/receive/FUDISS_thesis_000000001757>.
- Salomon, M., Kroon, L.G., Kuik, R., van Wassenhove, L.N., 1991. Some extensions of the discrete lot sizing and scheduling problem. *Management Science* 37, 801–812.
- Salomon, M., Kuik, R., van Wassenhove, L.N., 1993. Statistical search methods for lot sizing problems. *Annals of Operations Research* 41, 453–468.
- Stadtler, H., 1996. Mixed integer programming model formulations for dynamic multi-item multi-level capacitated lot sizing. *European Journal of Operational Research* 94, 561–581.
- Stadtler, H., 1997. Reformulations of the shortest route model for dynamic multi-item multi-level capacitated lot sizing. *Operations Research Spektrum* 19, 87–96.
- Stadtler, H., 2003. Multilevel lot sizing with setup times and multiple constrained resources: internally rolling schedules with lot-sizing windows. *Operations Research* 51, 487–502.
- Stadtler, H., Sürie, C., 2000. Description of MLCLSP test instances. Technical report, Technische Universität Darmstadt. <http://www.bwl.tu-darmstadt.de/bwl1/forschung/ti_mlclsp/ti_mlclsp.php?FG=bwl1>.
- Sürie, C., Stadtler, H., 2003. The capacitated lot-sizing problem with linked lot sizes. *Management Science* 49, 1039–1054.
- Tempelmeier, H., 2008. *Material-Logistik: Modelle und Algorithmen für die Produktionsplanung und -steuerung in Advanced Planning-Systemen*, seventh ed Springer, Berlin.
- Tempelmeier, H., Derstroff, M., 1993. Mehrstufige Mehrprodukt-Losgrößenplanung bei beschränkten Ressourcen und genereller Erzeugnisstruktur. *Operations Research Spektrum* 15, 63–73.
- Tempelmeier, H., Derstroff, M., 1996. A Lagrangean-based heuristic for dynamic multilevel multiitem constrained lot sizing with setup times. *Management Science* 42, 738–757.
- Tempelmeier, H., Helber, S., 1994. A heuristic for dynamic multi-item multi-level capacitated lot sizing for general product structures. *European Journal of Operational Research* 75, 296–311.
- Wagner, H.M., Whitin, T.M., 1958. Dynamic version of the economic lot size model. *Management Science* 5, 89–96.
- Wolsey, L.A., 1995. Progress with single-item lot-sizing. *European Journal of Operational Research* 86, 395–401.
- Xie, J., Dong, J., 2002. Heuristic genetic algorithms for general capacitated lot-sizing problems. *Computers and Mathematics with Applications* 44, 263–276.